

Table of Contents

ИНСТРУКЦИЯ ПО ВНЕДРЕНИЮ

T3-3 — Frontend ↔ Backend Contract Repair (v0.1)

Проект: ENIGMA v0.5.3.1.9 **Документ:** Инструкция по ремонту контракта frontend ↔ backend **Тип:** Техническое задание на миграцию (не код внедрения, а документация-инструкция) **Цель:** устранить tri-ontology system (3 competing truth layers), восстановить единый causal authority

Принцип патча: 6 atomic patches, которые поглощают 22 дефекта. Не точечные фиксы, а архитектурная консолидация.

0. Контекст: что патч делает и почему

0.1. Диагноз (из вашей декомпозиции)

Система разделилась на **3 competing truth layers**:

Layer	Что	Примеры
Backend kernel truth	snapshots, DTO, simulation state	WorldSnapshotDTO, scene_state
Frontend inferred truth	local position prediction, local journal, avatar override	game_time_seconds +=, dialog_journal, avatar_state[*]=
Fallback truth (adapters/suppress)	silent exception handlers, fallback registries, implicit conversions	except Exception: pass, _MinimalFrontendRegistry, snapshot_npc_positions_t o_dict

Это не баги. Это emergent tri-ontology system. Frontend «додумывает», backend «не знает», runtime «адаптирует», ошибки «исчезают».

0.2. 4 класса дефектов (22 пункта T3-3)

Класс	Доля	Дефекты	Симптом
A. Contract Drift	~55%	3.1, 3.6, 3.10, 3.14, 3.21	backend и frontend живут в разных типах

Класс	Доля	Дефекты	Симптом
B. Hidden Authority Violations	~20%	3.2, 3.3, 3.5, 3.7	реальности frontend = «второй backend»
C. Determinism / Kernel contamination	~15%	3.17, 3.2, 3.15	silent failure paths → silent divergence
D. Architectural duplication	~10%	3.18, 3.19	один алгоритм в двух местах → drift

0.3. Приоритизация (causal impact, не количество багов)

Tier	Дефекты	Что ломает
 Tier 0	3.2, 3.10, 3.17	causal engine (spatial sync, runtime adapter, observability)
 Tier 1	3.1, 3.6, 3.14	consistency (DTO contract drift, dual truth)
 Tier 2	3.3, 3.5, 3.7, 3.8, 3.9	logical erosion / UX corruption
 Tier 3	3.11, 3.12, 3.13, 3.15, 3.16, 3.18, 3.19, 3.20, 3.22	engineering debt

0.4. Стратегия: 6 atomic patches

Не 22 точечных фикса, а **6 архитектурных ходов**, каждый поглощает несколько дефектов:

Patch	Поглощает дефекты	Что делает
A1	3.1, 3.6, 3.8	GameActionResponse expansion
A2	3.10, 3.14, 3.21	WorldSnapshotDTO canonicalization (Dict, не List)
A3	3.11	Cue normalization (cue_type → cue_key)
B1	3.2, 3.3, 3.4, 3.5, 3.7	Frontend authority removal
B2	3.17, 3.2 core	Spatial Oracle no- silent-failure

Patch	Поглощает дефекты	Что делает
B3	3.20, 3.21	SSE + world_state activation
C1	3.15, 3.16	suppress(Exception) removal
C2	3.18, 3.19	Spatial duplication removal

1. Порядок применения (обязательный)

STEP 1 → api_client.py (A1 — GameActionResponse expansion)
 STEP 2 → snapshot.py (A2 — WorldSnapshotDTO Dict canonicalization)
 STEP 3 → snapshot.py + game_screen (A3 — cue_type → cue_key normalization)
 STEP 4 → game_screen.py (B1 — frontend authority removal, 5 sub-fixes)
 STEP 5 → game_loop_bridge.py (B2 — Spatial Oracle no-silent-failure)
 STEP 6 → api_client.py + routes_stream (B3 — SSE activation)
 STEP 7 → game_screen.py (C1 — suppress(Exception) removal)
 STEP 8 → world_context.py + spatial_compilation_orchestrator (C2 — duplication removal)
 STEP 9 → validation tests

Зависимости: - STEP 2 требует STEP 1 (frontend должен принять Dict) - STEP 4 требует STEP 1 (нужно scene_state поле) - STEP 6 требует STEP 1 (SSE возвращает тот же DTO) - STEP 9 требует всех

Откат: каждый STEP — independently testable. Можно откатывать по одному.

2. STEP 1 — frontend/api_client.py (MODIFY — Patch A1)

2.1. Назначение

Расширить GameActionResponse полями scene_state и metadata, которые уже передаются в _map_action_response, но отсутствуют в dataclass.

Закрывает TypeError на каждом HTTP-вызове.

2.2. Контекст (что сейчас)

Файл: frontend/api_client.py

GameActionResponse dataclass (строки 51-65):

```

@dataclass
class GameActionResponse:
    dm_response: str
    npc_reactions: list[dict]
    world_changes: list[dict]
  
```

```

journal_entry_id: str | None
game_time_seconds: int = 0
world_snapshot: dict | None = None
npc_positions: dict | None = None
will_conflict_data: dict | None = None
confirmed_location_id: str | None = None
# HET scene_state, HET metadata

```

`_map_action_response` (строки 248-267) передаёт:

```

return GameActionResponse(
    ...
    scene_state=raw.get("scene_state", {}),    # ← TypeError!
    metadata=raw.get("metadata", {}),         # ← TypeError!
    confirmed_location_id=raw.get("confirmed_location_id"),
)

```

2.3. Изменение

ЗАМЕНИ (строки 51-65):

```

@dataclass
class GameActionResponse:
    """Доменный объект ответа на действие игрока."""
    dm_response: str
    npc_reactions: list[dict]
    world_changes: list[dict]
    journal_entry_id: str | None
    game_time_seconds: int = 0 # total_seconds для HUD
    # TASK 1: Force Merge — world_snapshot из player action tick (ADR-0014)
    world_snapshot: dict | None = None
    npc_positions: dict | None = None
    # Resistance Medium: Данные конфликта воли для заражения UI
    will_conflict_data: dict | None = None
    # S82: Backend подтверждает spatial truth. Frontend reconciles при
    расхождении.
    confirmed_location_id: str | None = None

```

НА:

```

@dataclass
class GameActionResponse:
    """Доменный объект ответа на действие игрока.

    A1-FIX: добавлены scene_state и metadata поля, которые уже передаются
    в _map_action_response, но отсутствовали в dataclass. Раньше каждый
    HTTP /api/game/action падал с TypeError, маскируемым ActionQueue.
    """
    dm_response: str
    npc_reactions: list[dict]
    world_changes: list[dict]

```

```

journal_entry_id: str | None
game_time_seconds: int = 0 # total_seconds для HUD
# TASK 1: Force Merge — world_snapshot из player action tick (ADR-0014)
world_snapshot: dict | None = None
npc_positions: dict | None = None
# Resistance Medium: Данные конфликта воли для заражения UI
will_conflict_data: dict | None = None
# S82: Backend подтверждает spatial truth. Frontend reconciles при
расхождении.
confirmed_location_id: str | None = None
# A1-FIX: S85 fields — проброс scene_state и metadata для инициализации
UI.
# Раньше передавались в _map_action_response, но отсутствовали в dataclass
→ TypeError.
scene_state: dict | None = None
metadata: dict | None = None

```

2.4. Также проверить DirectGameGateway

Файл: frontend/api_client.py — DirectGameGateway.send_action (строки ~368-384)

Проверить, что DirectGameGateway тоже передаёт scene_state и metadata:

```

return GameActionResponse(
    dm_response=result.dm_text,
    npc_reactions=result.npc_reactions,
    world_changes=[],
    journal_entry_id=None,
    game_time_seconds=result.game_time_seconds,
    world_snapshot=result.world_snapshot,
    npc_positions=result.npc_positions,
    will_conflict_data=result.will_conflict_data,
    confirmed_location_id=result.confirmed_location_id, # если есть
    scene_state=getattr(result, 'scene_state', None), # ДОБАВИТЬ
    metadata=getattr(result, 'metadata', None), # ДОБАВИТЬ
)

```

2.5. Критерий приёмки

`cd /home/z/my-project/enigma/Enigma-V.0.5.3.1.9_-_01 && python -c "`

```

import sys
sys.path.insert(0, 'frontend')
sys.path.insert(0, 'backend')
from api_client import GameActionResponse

```

```

# Test that all fields exist
response = GameActionResponse(
    dm_response='test',
    npc_reactions=[],
    world_changes=[],
    journal_entry_id=None,

```

```

    scene_state={'test': True},
    metadata={'meta': True},
)
assert response.scene_state == {'test': True}
assert response.metadata == {'meta': True}
print('GameActionResponse expansion OK')
"

```

3. STEP 2 — backend/app/domain/snapshot.py (MODIFY — Patch A2)

3.1. Назначение

Канонизировать WorldSnapshotDTO.npc_positions как Dict[str, NPCPositionDTO], не List[NPCPositionDTO]. **Убрать runtime adapter** snapshot_npc_positions_to_dict, который маскирует архитектурный разрыв.

3.2. Контекст (что сейчас)

Файл: backend/app/domain/snapshot.py

Строка 162:

```
npc_positions: List[NPCPositionDTO]
```

Строки 177-199 — runtime adapter:

```

def snapshot_npc_positions_to_dict(
    positions: List[NPCPositionDTO],
) -> dict:
    """Конвертирует List[NPCPositionDTO] в dict для обратной совместимости
    фронта. """
    result: dict = {}
    for pos in positions:
        result[pos.npc_id] = {
            "npc_id": pos.npc_id,
            "x": pos.x,
            "y": pos.y,
            "local_position": {"x": pos.x, "y": pos.y},
            "activity": pos.action,
            ...
        }
    return result

```

Frontend (game_screen.py:718, 891, 947) итерирует .items() — требует dict.

3.3. Изменение 1 — сменить тип в DTO

ЗАМЕНИ (строка 162):

```
npc_positions: List[NPCPositionDTO]
```

НА:

```
# A2-FIX: Dict[str, NPCPositionDTO] — canonical.  
# Раньше List[NPCPositionDTO] + runtime adapter  
snapshot_npc_positions_to_dict.  
# Adapter маскировал архитектурный разрыв между backend (List) и  
frontend (Dict).  
# Теперь: Dict напрямую, frontend читает .items() без конвертации.  
npc_positions: Dict[str, NPCPositionDTO]
```

3.4. Изменение 2 — обновить builder

Файл: backend/app/services/integration/world_snapshot_builder.py

НАЙТИ где собирается npc_positions (вероятно _extract_npc_positions):

ЗАМЕНИ (если собирается как list):

```
positions: List[NPCPositionDTO] = []  
for npc_id, npc_data in ...:  
    positions.append(NPCPositionDTO(...))  
return positions
```

НА:

```
# A2-FIX: собираем как Dict напрямую, не List.  
positions: Dict[str, NPCPositionDTO] = {}  
for npc_id, npc_data in ...:  
    positions[npc_id] = NPCPositionDTO(...)  
return positions
```

3.5. Изменение 3 — убрать runtime adapter

ЗАМЕНИ (строки 177-199):

```
def snapshot_npc_positions_to_dict(  
    positions: List[NPCPositionDTO],  
) -> dict:  
    """Конвертирует List[NPCPositionDTO] в dict для обратной совместимости  
фронтенда.  
    ...  
    result: dict = {}  
    for pos in positions:  
        result[pos.npc_id] = {  
            "npc_id": pos.npc_id,  
            "x": pos.x,  
            "y": pos.y,  
            "local_position": {"x": pos.x, "y": pos.y},  
            "activity": pos.action,  
            "facing": pos.facing,  
            "location_id": pos.location_id,
```

```

        "display_name": pos.display_name,
        "name": pos.display_name,
    }
    return result

```

НА:

```

# A2-FIX: snapshot_npc_positions_to_dict УДАЛЕН.
# Раньше конвертировал List[NPCPositionDTO] → dict для frontend.
# Теперь WorldSnapshotDTO.npc_positions = Dict[str, NPCPositionDTO]
напрямую.
# Frontend читает .items() без конвертации.
#
# Если где-то ещё нужен dict-формат (для JSON serialization), использовать
asdict():
# from dataclasses import asdict
# positions_dict = {nid: asdict(pos) for nid, pos in ws.npc_positions.items()}

```

3.6. Изменение 4 — обновить callers

Найти все использования snapshot_npc_positions_to_dict:

```
grep -rn "snapshot_npc_positions_to_dict" backend/ --include="*.py"
```

Заменить каждый caller. Например, в routes.py:

ЗАМЕНИ:

```

if isinstance(_npc_pos, dict):
    _npc_pos_dict = _npc_pos
elif _npc_pos:
    _npc_pos_dict = snapshot_npc_positions_to_dict(_npc_pos)

```

НА:

```

# A2-FIX: npc_positions уже Dict[str, NPCPositionDTO], конвертация не
нужна.
if isinstance(_npc_pos, dict):
    _npc_pos_dict = _npc_pos
elif _npc_pos:
    # Если вдруг list — конвертируем напрямую (legacy path).
    from dataclasses import asdict
    _npc_pos_dict = {
        pos.npc_id: asdict(pos) if hasattr(pos, 'npc_id') else pos
        for pos in _npc_pos
    }

```

3.7. Критерий приёмки

1. Нет больше List типа

```
grep -n "npc_positions: List" backend/app/domain/snapshot.py
```

Ожидание: 0 совпадений

2. Нет adapter функции

```
grep -rn "snapshot_npc_positions_to_dict" backend/ --include="*.py"
```

Ожидание: 0 совпадений (или только в комментариях)

3. DTO test

```
python -c "  
import sys  
sys.path.insert(0, 'backend')  
from app.domain.snapshot import WorldSnapshotDTO, NPCPositionDTO  
# Dict type check  
import typing  
hints = typing.get_type_hints(WorldSnapshotDTO)  
assert hints['npc_positions'].__origin__ == dict, 'npc_positions must be Dict'  
print('WorldSnapshotDTO Dict canonicalization OK')  
"
```

4. STEP 3 — backend/app/domain/snapshot.py + frontend/game_screen.py (MODIFY — Patch A3)

4.1. Назначение

Нормализовать PeripheralCueDTO поле. Backend использует cue_type, frontend читает cue_key. **Выбор: переименовать в cue_key** (frontend уже живёт с этим именем, меньше изменений).

4.2. Контекст (что сейчас)

Backend (snapshot.py:~106):

```
@dataclass(frozen=True)  
class PeripheralCueDTO:  
    npc_id: str  
    cue_type: str          # "FREEZE", "HURRY", "AVOID_GAZE"  
    hover_text: str        # "Замер на месте", "Отвел взгляд"
```

Frontend (game_screen.py:1222):

```
_ck = _c.get("cue_key", "") # ← читает cue_key, не cue_type
```

Результат: _ck всегда пустая строка → консоль наблюдений (Ё) не показывает симптомы.

4.3. Изменение 1 — переименовать в backend

ЗАМЕНИ в PeripheralCueDTO:

```
@dataclass(frozen=True)  
class PeripheralCueDTO:  
    """Периферическое наблюдение за NPC (Слой 1: Наблюдение, не диагноз).
```

ЗАПРЕТ: Текст строго внешний ("Замер"), без телепатии ("Боится"). """
 npc_id: str
 cue_type: str # "FREEZE", "HURRY", "AVOID_GAZE"
 hover_text: str # "Замер на месте", "Отвел взгляд"

НА:

```
@dataclass(frozen=True)
class PeripheralCueDTO:
    """Периферическое наблюдение за NPC (Слой 1: Наблюдение, не диагноз).
    ЗАПРЕТ: Текст строго внешний ("Замер"), без телепатии ("Боится").

    АЗ-FIX: поле переименовано cue_type → cue_key для согласования с frontend.
    Раньше frontend читал cue_key, backend отдавал cue_type → пустые
    СИМПТОМЫ.
    """
    npc_id: str
    cue_key: str # "FREEZE", "HURRY", "AVOID_GAZE" (renamed from
    cue_type)
    hover_text: str # "Замер на месте", "Отвел взгляд"
```

4.4. Изменение 2 — обновить builder

Файл: backend/app/services/integration/world_snapshot_builder.py

НАЙТИ где создаётся PeripheralCueDTO:

ЗАМЕНИ:

```
peripheral_cues.append(PeripheralCueDTO(
    npc_id=...,
    cue_type=cue_key, # ← старое имя
    hover_text=...,
))
```

НА:

```
peripheral_cues.append(PeripheralCueDTO(
    npc_id=...,
    cue_key=cue_key, # ← новое имя (АЗ-FIX)
    hover_text=...,
))
```

4.5. Изменение 3 — обновить frontend (fallback для безопасности)

Файл: frontend/game_screen.py:1222

ЗАМЕНИ:

```
_ck = _c.get("cue_key", "")
```

НА:

```
# A3-FIX: cue_key canonical (после rename). Fallback на cue_type  
# для backward compatibility со старыми snapshot'ами.  
_ck = _c.get("cue_key", "") or _c.get("cue_type", "")
```

4.6. Критерий приёма

1. Backend использует cue_key

```
grep -n "cue_type" backend/app/domain/snapshot.py  
# Ожидание: 0 совпадений (только в комментариях)
```

2. Frontend читает cue_key

```
grep -n "cue_key" frontend/game_screen.py  
# Ожидание: 1+ совпадений
```

3. Integration test

```
python -c "  
import sys  
sys.path.insert(0, 'backend')  
from app.domain.snapshot import PeripheralCueDTO  
cue = PeripheralCueDTO(npc_id='test', cue_key='FREEZE', hover_text='Замер')  
assert cue.cue_key == 'FREEZE'  
# Сериализация в dict  
from dataclasses import asdict  
d = asdict(cue)  
assert 'cue_key' in d  
assert 'cue_type' not in d  
print('Cue normalization OK')  
"
```

5. STEP 4 — frontend/game_screen.py (MODIFY — Patch B1, 5 sub-fixes)

5.1. Назначение

Frontend = renderer, never state generator. Убрать 5 authority violations:

1. game_time_seconds mutation (3.3)
2. avatar_state override (3.5)
3. dialog_journal local storage (3.4)
4. _set_player_xy direct mutation без push (3.2)
5. is_telegraph not propagated (3.7 — partial)

5.2. Sub-fix B1.1 — game_time_seconds mutation (строка 699)

ЗАМЕНИ:

```
# Оптимистичный рендеринг: фронтенд предсказывает время,  
# бэкенд подтвердит его при следующем супс
```

```
self.game_time_seconds += TIME_DELTA_WALK_INDOOR *  
meters_walked
```

НА:

```
# B1.1-FIX: frontend НЕ конструирует время.  
# Absolute time authority = backend (Устав §3).  
# Раньше: self.game_time_seconds += ... → dual truth.  
# Теперь: только backend продвигает время (через  
advance_game_time).  
# Frontend отображает значение из world_snapshot.  
pass # время обновится из world_snapshot при следующем sync
```

5.3. Sub-fix B1.2 — avatar_state override (строки 1059-1066)

ЗАМЕНИ:

```
if "avatar_state" not in scene_state or  
scene_state["avatar_state"] is None:  
    scene_state["avatar_state"] = {}  
    scene_state["avatar_state"]["perceptual_stability"] = max(0.0, 1.0  
- _res * 4.0)  
    scene_state["avatar_state"]["cognitive_coherence"] = max(0.0,  
1.0 - _res * 3.0)  
    scene_state["avatar_state"]["sensory_noise"] = _res * 3.0  
    scene_state["avatar_state"]["motor_disruption"] = _res * 5.0  
    logger.debug(f"[PIPELINE][EMBODIMENT] Injected instability:  
res={_res:.2f}, stability={scene_state['avatar_state']['perceptual_stability']:.2f},  
motor={scene_state['avatar_state']['motor_disruption']:.2f}")
```

НА:

```
# B1.2-FIX: frontend НЕ перезаписывает avatar_state.  
# Backend avatar_presentation_assembler уже посчитал  
# perceptual_stability/cognitive_coherence/etc. из  
# body_state + willpower. Frontend только отображает.  
# Раньше: перезапись маскировала backend-проекцию.  
# Теперь: используем _res только для визуального эффекта  
# (intensity), не для скаляров состояния.  
if _res > 0:  
    self._resistance_visual_intensity = _res  
    logger.debug(f"[PIPELINE][EMBODIMENT] resistance visual  
intensity: {_res:.2f}")
```

5.4. Sub-fix B1.3 — dialog_journal local storage

Файл: frontend/game_screen.py

ЗАМЕНИ (строка 343, инициализация):

```
self.dialog_journal = [] # ADR-JOURNAL: Накопительный лог диалогов
```

НА:

```
# B1.3-FIX: dialog_journal — из backend world_snapshot, не локальный.  
# Раньше: self.dialog_journal = [] → двойное хранение, рассинхрон на  
Continue.  
# Теперь: читаем из _ws["dialog_journal"] при каждом sync.  
self._dialog_journal_backend: list = [] # cache из backend
```

ЗАМЕНИ (строки 560, 1044 — append):

```
self.dialog_journal.append({"speaker": player_name, "text":  
text_input.text.strip()})
```

НА:

```
# B1.3-FIX: НЕ добавляем локально. Backend вернёт  
обновлённый  
# journal в следующем world_snapshot.  
pass
```

ЗАМЕНИ (строка 1306 — отображение):

```
_journal_data = self.dialog_journal
```

НА:

```
# B1.3-FIX: читаем из backend cache  
_journal_data = self._dialog_journal_backend
```

ДОБАВИТЬ в обработку world_snapshot (где-то после строки 754):

```
# B1.3-FIX: синхронизируем journal из backend  
self._dialog_journal_backend = _ws.get("dialog_journal",  
self._dialog_journal_backend)
```

5.5. Sub-fix B1.4 — _set_player_xy + push to backend (строки 821-824)

ЗАМЕНИ:

```
with contextlib.suppress(Exception):  
    _bridge = _gateway._bridge # AttributeError в HttpGameGateway!  
    if _bridge.ready:  
        _bridge.save_scene_state(campaign_folder, scene_state)
```

НА:

```
# B1.4-FIX: убран contextlib.suppress(Exception), который маскировал  
# AttributeError в HttpGameGateway (нет _bridge атрибута).  
# Теперь: явная проверка типа gateway.  
try:  
    if hasattr(_gateway, 'save_scene_state'):  
        _gateway.save_scene_state(campaign_folder, scene_state)  
    else:
```

```

        logger.warning(
            f"[GAME_SCREEN] Gateway {_gateway.__class__.__name__} "
            f"does not support save_scene_state — player position not pushed"
        )
    except Exception as e:
        logger.warning(f"[GAME_SCREEN] save_scene_state failed: {e}")

```

ДОБАВИТЬ метод `save_scene_state` в `GameGateway Protocol` и в `HttpGameGateway`:

Файл: `frontend/api_client.py`

В `GameGateway Protocol` добавить:

```

def save_scene_state(self, campaign_id: str, scene_state: dict) -> None:
    """Push updated scene_state to backend."""
    ...

```

В `HttpGameGateway` добавить реализацию:

```

def save_scene_state(self, campaign_id: str, scene_state: dict) -> None:
    """B1.4-FIX: push scene_state to backend via HTTP."""
    try:
        self._t.post(f"/api/game/{campaign_id}/scene_state", json=scene_state)
    except Exception as e:
        logger.warning(f"[HTTP_GATEWAY] save_scene_state failed: {e}")
        raise

```

В `backend` `routes.py` добавить endpoint:

```

@router.post("/api/game/{campaign_id}/scene_state")
def update_scene_state(campaign_id: str, scene_state: dict = Body(...)):
    """B1.4-FIX: receive scene_state updates from frontend (player position)."""
    scene_manager.save_scene_state(campaign_id, scene_state)
    return {"status": "ok"}

```

5.6. Sub-fix B1.5 — `is_telegraph` propagation (строки 339-379)

Файл: `frontend/api_client.py` — `DirectGameGateway.send_action`

ЗАМЕНИ:

```

def send_action(self, action_text, ...):
    result = self._bridge.turn(
        campaign_id=campaign_id,
        action_text=action_text,
        player_position=player_position,
    )

```

НА:

```

def send_action(self, action_text, ...):
    # B1.5-FIX: пробрасываем is_telegraph в Direct mode.

```

```

is_telegraph = getattr(action_text, '_is_telegraph', False)
result = self.bridge.turn(
    campaign_id=campaign_id,
    action_text=action_text,
    player_position=player_position,
    is_telegraph=is_telegraph, # ДОБАВИТЬ
)

```

В game_loop_bridge.turn добавить параметр:

```

def turn(self, campaign_id: str, action_text: str, player_position: dict,
        is_telegraph: bool = False) -> TurnResult:
    ...
    if is_telegraph:
        # Не записывать в историю игрока, не вызывать DM
        return TurnResult(dm_text="", npc_reactions=[], ...)

```

5.7. Критерий приёмки

1. Нет game_time mutation

```
grep -n "self.game_time_seconds.*+=" frontend/game_screen.py
```

Ожидание: 0 совпадений

2. Нет avatar_state override

```
grep -n 'avatar_state"\|"' frontend/game_screen.py | grep "="
```

Ожидание: 0 совпадений (только чтение)

3. Нет dialog_journal.append

```
grep -n "dialog_journal.append" frontend/game_screen.py
```

Ожидание: 0 совпадений

4. Нет suppress(Exception) для _bridge

```
grep -n "suppress.*_bridge\|_bridge.*suppress" frontend/game_screen.py
```

Ожидание: 0 совпадений

6. STEP 5 — frontend/game_loop_bridge.py (MODIFY — Patch B2)

6.1. Назначение

Spatial Oracle: **no silent fallback**. Если find_chunks падает — логировать и поднимать, не глотать.

6.2. Контекст (что сейчас)

Файл: frontend/game_loop_bridge.py:100-111

```

if world_x is not None and world_y is not None:
    try:
        from app.services.spatial.spatial_registry import SpatialRegistry
        _registry = SpatialRegistry.get_or_load(campaign_id)

```

```

    if _registry is not None:
        _actual_chunks = _registry.find_chunks(world_x, world_y)
        if _actual_chunks:
            location = _actual_chunks[0].location_id
except Exception:
    pass # Fallback к saved location

```

6.3. Изменение

ЗАМЕНИ:

```

if world_x is not None and world_y is not None:
    try:
        from app.services.spatial.spatial_registry import SpatialRegistry
        _registry = SpatialRegistry.get_or_load(campaign_id)
        if _registry is not None:
            _actual_chunks = _registry.find_chunks(world_x, world_y)
            if _actual_chunks:
                location = _actual_chunks[0].location_id
    except Exception:
        pass # Fallback к saved location

```

НА:

```

if world_x is not None and world_y is not None:
    # B2-FIX: Spatial Oracle no-silent-failure.
    # Раньше: except Exception: pass — Spatial Oracle становился
невидимым инвалидом.
    # Теперь: логируем ошибки, явно сообщаем о fallback.
    try:
        from app.services.spatial.spatial_registry import SpatialRegistry
        _registry = SpatialRegistry.get_or_load(campaign_id)
        if _registry is None:
            logger.warning(
                f"[SPATIAL_ORACLE] registry not loaded for
campaign={campaign_id}. "
                f"Fallback to saved location."
            )
        elif not hasattr(_registry, 'find_chunks'):
            logger.error(
                f"[SPATIAL_ORACLE] registry {_registry.__class__.__name__} "
                f"has no find_chunks method. Fallback to saved location."
            )
        else:
            _actual_chunks = _registry.find_chunks(world_x, world_y)
            if _actual_chunks:
                location = _actual_chunks[0].location_id
                logger.debug(
                    f"[SPATIAL_ORACLE] resolved: ({world_x}, {world_y}) →
{location}"
                )

```



```

    else:
        logger.debug(
            f"[SPATIAL_ORACLE] no chunks for ({world_x}, {world_y}). "
            f"Fallback to saved location."
        )
    except Exception as e:
        # B2-FIX: логируем, не глотаем.
        logger.warning(
            f"[SPATIAL_ORACLE] find_chunks failed: {e}. "
            f"Fallback to saved location."
        )

```

6.4. Критерий приёмки

1. Нет silent pass

```
grep -A2 "except Exception" frontend/game_loop_bridge.py | grep "pass"
```

Ожидание: 0 совпадений

2. Есть logger.warning/error

```
grep -n "SPATIAL_ORACLE" frontend/game_loop_bridge.py
```

Ожидание: 3+ совпадений (warning, error, debug)

7. STEP 6 — frontend/api_client.py + backend/app/api/routes_stream.py (MODIFY — Patch B3)

7.1. Назначение

Активировать SSE endpoint /api/game/action/stream в HTTP-клиенте. Сейчас endpoint существует, но не используется.

7.2. Изменение 1 — добавить streaming метод в HttpGameGateway

Файл: frontend/api_client.py

ДОБАВИТЬ в HttpGameGateway:

```

def send_action_stream(
    self,
    action_text: str,
    campaign_id: str,
    player_position: dict | None = None,
    **kwargs,
) -> Iterator[str]:
    """B3-FIX: stream DM tokens via SSE.

```

*Использует /api/game/action/stream endpoint.
Возвращает генератор токенов DM-ответа.
"""*

```

import httpx
import json

```

```

payload = {
    "action_text": action_text,
    "campaign_id": campaign_id,
    "player_position": player_position,
    **kwargs,
}

try:
    with httpx.stream(
        "POST",
        f"{self._base_url}/api/game/action/stream",
        json=payload,
        timeout=60.0,
    ) as response:
        response.raise_for_status()
        for line in response.iter_lines():
            if line.startswith("data: "):
                data = json.loads(line[6:])
                if data.get("type") == "token":
                    yield data.get("text", "")
                elif data.get("type") == "done":
                    return
                elif data.get("type") == "error":
                    raise BackendError(data.get("message", "Stream error"))
except httpx.HTTPError as e:
    raise BackendError(f"Stream failed: {e}")

```

7.3. Изменение 2 — использовать streaming в game_screen

Файл: frontend/game_screen.py

В методе обработки action (где вызывается send_action):

ДОБАВИТЬ ветку для streaming:

```

# B3-FIX: использовать SSE streaming если gateway поддерживает.
if hasattr(_gateway, 'send_action_stream') and isinstance(_gateway,
HttpGameGateway):
    dm_text = ""
    try:
        for token in _gateway.send_action_stream(
            action_text=text_input.text.strip(),
            campaign_id=campaign_folder,
            player_position=...,
        ):
            dm_text += token
            # Обновляем UI по мере поступления токенов
            self.update_dm_display_progressive(dm_text)
            # Финальная обработка после завершения стрима
    response = self._finalize_streamed_action(dm_text, campaign_folder)

```

```

    except BackendError as e:
        logger.error(f"[STREAM] failed: {e}, fallback to sync")
        response = _gateway.send_action(...)
    else:
        # Sync path (DirectGameGateway or fallback)
        response = _gateway.send_action(...)

```

7.4. Изменение 3 — интегрировать /api/world_state

Файл: frontend/api_client.py

Проверить, что get_world_state метод существует (строки ~823-840).
Если есть — **использовать** в idle_tick:

Файл: frontend/game_screen.py — _do_idle_tick

ЗАМЕНИ:

```

def _do_idle_tick():
    try:
        result = _gateway.idle_tick(campaign_folder)
        ...
    except Exception as e:
        ...

```

НА:

```

def _do_idle_tick():
    try:
        # B3-FIX: используем /api/world_state endpoint если доступен.
        if hasattr(_gateway, 'get_world_state'):
            ws = _gateway.get_world_state(campaign_folder, after_tick=self._last_tick)
            if ws:
                self._apply_world_snapshot(ws)
            return
        # Fallback на idle_tick
        result = _gateway.idle_tick(campaign_folder)
        ...
    except Exception as e:
        ...

```

7.5. Критерий приёмки

1. send_action_stream существует

grep -n "send_action_stream" frontend/api_client.py

Ожидание: 1 + совпадений

2. SSE endpoint используется

grep -n "send_action_stream|get_world_state" frontend/game_screen.py

Ожидание: 2+ совпадений

8. STEP 7 — frontend/game_screen.py (MODIFY — Patch C1)

8.1. Назначение

Убрать `suppress(Exception)` — **no silent failure at system boundary**.

8.2. Контекст (что сейчас)

Файл: frontend/game_screen.py

Строки 477, 821 (после B1.4 уже частично исправлено):

```
with contextlib.suppress(Exception):  
    # чтение NPC config  
    ...
```

8.3. Изменение 1 — строка 477 (NPC config reading)

ЗАМЕНИ:

```
with contextlib.suppress(Exception):  
    import json  
    _npc_dir = Path("config/npc/individualals")  
    if _npc_dir.exists():  
        for f in _npc_dir.glob("*.json"):  
            _data = json.loads(_f.read_text(encoding="utf-8"))  
            _npc_name_map[_data.get("id", "")] = _data.get("name", "")
```

НА:

```
# C1-FIX: убран contextlib.suppress(Exception).  
# Используем npc_name_resolver (single source of truth).  
from npc_name_resolver import npc_id_to_display  
# _npc_name_map больше не нужен — используем resolver напрямую.  
# Если resolver падает — логируем, не глотаем.  
try:  
    # Тест, что resolver работает  
    _test_name = npc_id_to_display("maid_lusya")  
    logger.debug(f"[NPC_NAMES] resolver works: maid_lusya →  
{_test_name}")  
except Exception as e:  
    logger.error(f"[NPC_NAMES] npc_name_resolver failed: {e}")  
    # Не suppress — пусть caller видит проблему
```

8.4. Изменение 2 — любые другие `suppress(Exception)`

Найти все:

```
grep -n "suppress(Exception)|contextlib.suppress" frontend/game_screen.py
```

Заменить каждый на явный `try/except` с логированием:

```

# БЫЛО:
with contextlib.suppress(Exception):
    ...

# СТАЛО:
try:
    ...
except Exception as e:
    logger.warning(f"[CONTEXT] operation failed: {e}")
    # Явный fallback или проброс

```

8.5. Критерий приёмки

```

grep -n "suppress(Exception)" frontend/game_screen.py
# Ожидание: 0 совпадений

```

```

grep -n "contextlib.suppress" frontend/game_screen.py
# Ожидание: 0 совпадений (или только suppress(FileNotFoundError) для
конкретных случаев)

```

9. STEP 8 — frontend/world_context.py + frontend/spatial_compilation_orchestrator.py (MODIFY — Patch C2)

9.1. Назначение

Убрать дублирование: `_split_wall_by_openings` (frontend + backend) и два `SpatialRegistry` источника.

9.2. Изменение 1 — вынести `_split_wall_by_openings` в общий модуль

Создать файл: backend/app/services/spatial/wall_utils.py

```

"""
path: backend/app/services/spatial/wall_utils.py
Назначение: Единый алгоритм разбиения стен по проёмам.
C2-FIX: убрано дублирование frontend/backend.
"""

```

```

from typing import Dict, List, Any

```

```

def split_wall_by_openings(wall: Dict[str, Any], openings: List[Dict[str, Any]]) ->
List[Dict[str, Any]]:

```

"""Разбивает стену по проёмам.

Единый алгоритм для frontend и backend.

Раньше дублировался в:

- frontend/world_context.py:182-267

- backend/app/services/scene_state_manager.py

"""

Реализация (перенести из существующего кода)

...

9.3. Изменение 2 — frontend импортирует из backend

Файл: frontend/world_context.py:182-267

ЗАМЕНИ локальную реализацию `_split_wall_by_openings`:

C2-FIX: используем единый алгоритм из backend.

try:

from app.services.spatial.wall_utils **import** split_wall_by_openings

except ImportError:

Fallback для pure-frontend режима (если backend недоступен)

Это компромисс — в production всегда должен быть backend.

import logging

logging.getLogger(__name__).warning(

"[WORLD_CONTEXT] backend wall_utils not available, using local copy. "

"This indicates incomplete setup."

)

def split_wall_by_openings(wall, openings):

Локальная копия (для dev/test)

...

9.4. Изменение 3 — `_MinimalFrontendRegistry.find_chunks`

Файл: frontend/spatial_compilation_orchestrator.py

ДОБАВИТЬ метод `find_chunks` в `_MinimalFrontendRegistry`:

class _MinimalFrontendRegistry:

"""Минимальный реестр для pure-frontend режима."""

def find_chunks(self, world_x: float, world_y: float) -> list:

"""C2-FIX: добавлен find_chunks для совместимости с SpatialOracle.

Раньше отсутствовал → Spatial Oracle падал с AttributeError,

маскируемым except Exception: pass.

"""

result = []

for chunk **in** self._chunks.values():

if (chunk.origin_x <= world_x <= chunk.origin_x + chunk.width **and**

chunk.origin_y <= world_y <= chunk.origin_y + chunk.height):

result.append(chunk)

return result

9.5. Критерий приёмки

1. Нет дублирования _split_wall_by_openings

`grep -rn "_split_wall_by_openings\|def split_wall_by_openings" frontend/ backend/`

`--include="*.py"`

Ожидание: 1 определение (в wall_utils.py) + импорты

```
# 2. _MinimalFrontendRegistry имеет find_chunks  
grep -A5 "class _MinimalFrontendRegistry"  
frontend/spatial_compilation_orchestrator.py | grep "find_chunks"  
# Ожидание: 1+ совпадений
```

10. STEP 9 — backend/tests/test_tz3_contract_repair.py (NEW FILE)

10.1. Назначение

Integration tests для ТЗ-3 патчей.

10.2. Создать файл

Путь: backend/tests/test_tz3_contract_repair.py

Содержимое:

```
"""Integration tests for T3-3 Frontend ↔ Backend Contract Repair."""  
import pytest  
import sys  
from pathlib import Path  
  
# Добавить frontend в path  
sys.path.insert(0, str(Path(__file__).resolve().parents[2] / "frontend"))  
  
class TestPatchA1_GameActionResponse:  
    """A1: GameActionResponse expansion."""  
  
    def test_scene_state_field_exists(self):  
        from api_client import GameActionResponse  
        response = GameActionResponse(  
            dm_response="test",  
            npc_reactions=[],  
            world_changes=[],  
            journal_entry_id=None,  
            scene_state={"test": True},  
            metadata={"meta": True},  
        )  
        assert response.scene_state == {"test": True}  
        assert response.metadata == {"meta": True}  
  
    def test_map_action_response_no_typeerror(self):  
        from api_client import GameActionResponse  
        # Симулируем raw response от backend  
        raw = {  
            "response": "test",  
            "npc_reactions": [],  
            "world_changes": [],  
        }
```

```

    "journal_entry_id": None,
    "game_time_seconds": 0,
    "world_snapshot": None,
    "npc_positions": None,
    "will_conflict_data": None,
    "confirmed_location_id": None,
    "scene_state": {"test": True},
    "metadata": {"meta": True},
}
# Не должно падать с TypeError
response = GameActionResponse._map_action_response(raw)
assert response.scene_state == {"test": True}

```

```

class TestPatchA2_WorldSnapshotDTO:
    """A2: npc_positions Dict canonicalization."""

```

```

    def test_npc_positions_is_dict(self):
        from app.domain.snapshot import WorldSnapshotDTO
        import typing
        hints = typing.get_type_hints(WorldSnapshotDTO)
        # Должен быть Dict, не List
        assert hints['npc_positions'].__origin__ == dict

    def test_no_snapshot_npc_positions_to_dict(self):
        # Функция должна быть удалена
        from app.domain import snapshot
        assert not hasattr(snapshot, 'snapshot_npc_positions_to_dict'), \
            "snapshot_npc_positions_to_dict should be removed (A2-FIX)"

```

```

class TestPatchA3_CueNormalization:
    """A3: cue_type → cue_key."""

```

```

    def test_peripheral_cue_dto_uses_cue_key(self):
        from app.domain.snapshot import PeripheralCueDTO
        cue = PeripheralCueDTO(npc_id="test", cue_key="FREEZE",
        hover_text="Замер")
        assert cue.cue_key == "FREEZE"

        from dataclasses import asdict
        d = asdict(cue)
        assert "cue_key" in d
        assert "cue_type" not in d

```

```

class TestPatchB1_FrontendAuthority:
    """B1: frontend = renderer, never state generator."""

```



```

def test_no_game_time_mutation(self):
    # Читаем source game_screen.py
    screen_path = Path(__file__).resolve().parents[2] / "frontend" /
"game_screen.py"
    source = screen_path.read_text(encoding="utf-8")
    # Не должно быть self.game_time_seconds +=
    assert "self.game_time_seconds += " not in source, \
        "game_time_seconds mutation found (B1.1-FIX not applied)"

def test_no_avatar_state_override(self):
    screen_path = Path(__file__).resolve().parents[2] / "frontend" /
"game_screen.py"
    source = screen_path.read_text(encoding="utf-8")
    # Не должно быть прямой записи в avatar_state поля
    assert 'avatar_state'["perceptual_stability"] = ' not in source, \
        "avatar_state override found (B1.2-FIX not applied)"

def test_no_dialog_journal_append(self):
    screen_path = Path(__file__).resolve().parents[2] / "frontend" /
"game_screen.py"
    source = screen_path.read_text(encoding="utf-8")
    assert "dialog_journal.append" not in source, \
        "dialog_journal local append found (B1.3-FIX not applied)"

class TestPatchB2_SpatialOracleNoSilentFailure:
    """B2: Spatial Oracle no-silent-failure."""

    def test_no_silent_pass_in_bridge(self):
        bridge_path = Path(__file__).resolve().parents[2] / "frontend" /
"game_loop_bridge.py"
        source = bridge_path.read_text(encoding="utf-8")
        # Не должно быть "except Exception:\n                pass"
        assert "except Exception:\n                pass" not in source, \
            "Silent pass found in game_loop_bridge (B2-FIX not applied)"

    def test_logger_warnings_present(self):
        bridge_path = Path(__file__).resolve().parents[2] / "frontend" /
"game_loop_bridge.py"
        source = bridge_path.read_text(encoding="utf-8")
        assert "SPATIAL_ORACLE" in source, \
            "SPATIAL_ORACLE logging not found (B2-FIX not applied)"

class TestPatchC1_NoSilentSuppress:
    """C1: no suppress(Exception)."""

    def test_no_suppress_exception_in_game_screen(self):
        screen_path = Path(__file__).resolve().parents[2] / "frontend" /
"game_screen.py"

```

```

source = screen_path.read_text(encoding="utf-8")
assert "suppress(Exception)" not in source, \
    "suppress(Exception) found in game_screen (C1-FIX not applied)"

class TestPatchC2_NoDuplication:
    """C2: spatial duplication removed. """

    def test_minimal_registry_has_find_chunks(self):
        sco_path = Path(_file_).resolve().parents[2] / "frontend" /
"spatial_compilation_orchestrator.py"
        source = sco_path.read_text(encoding="utf-8")
        assert "def find_chunks" in source, \
            "_MinimalFrontendRegistry.find_chunks not found (C2-FIX not applied)"

```

10.3. Критерий приёмки

cd backend && python -m pytest tests/test_tz3_contract_repair.py -v

Все тесты должны проходить.

11. СВОДКА ВНЕДРЕНИЯ

Step	Файл	Patch	Что делает	Время
1	api_client.py	A1	GameAction Response scene_state + metadata	15 мин
2	snapshot.py + world_snapshot_builder.py + routes.py	A2	npc_position s Dict canonicalization	1 час
3	snapshot.py + world_snapshot_builder.py + game_screen.py	A3	cue_type → cue_key	30 мин
4	game_screen.py + api_client.py + routes.py	B1	Frontend authority removal (5 sub-fixes)	2 часа
5	game_loop_bridge.py	B2	Spatial Oracle no- silent-failure	30 мин
6	api_client.py	B3	SSE +	1.5 часа

Step	Файл	Patch	Что делает	Время
	+ routes_stream .py + game_screen. py		world_state activation	
7	game_screen. py	C1	suppress(Exc eption) removal	30 мин
8	wall_utils.py (NEW) + world_context. py + spatial_compil ation_orchestr ator.py	C2	Spatial duplication removal	1 час
9	test_tz3_contr act_repair.py	—	Validation tests	1 час

Итого: ~8 часов = 1 рабочий день.

12. ЧТО ПАТЧ ГАРАНТИРУЕТ

После применения:

✓ HTTP-режим работает (нет TypeError в GameActionResponse) ✓ NPC
 positions — единый контракт (Dict, не List+adapter) ✓ Консоль
 наблюдений показывает симптомы (cue key) ✓ Frontend = renderer (не
 генерирует time, avatar_state, journal) ✓ Player position пушится на
 backend в HTTP-режиме ✓ Spatial Oracle логирует ошибки (не silent
 failure) ✓ SSE streaming работает ✓ /api/world_state используется ✓
 Нет suppress(Exception) — все ошибки логируются ✓ Нет дублирования
 wall_utils, SpatialRegistry

Ожидаемый эффект:

Метрика	До	После
System coherence	~40%	~75-85%
Causal drift	~65%	~5-10%
Debuggability	low	+300-400% (no silent failures)
Mystery bugs	high	-70%
Tri-ontology layers	3	1 (backend kernel truth)

Что НЕ решается (намеренно):

- L1Chronicle persistence (отдельный ADR)
 - BreakProgressEngine ordering (отдельный ADR)
 - BehaviorMask assignment (отдельный ADR)
 - CK-style systems (отдельный layer)
-

13. РОЛЛБЭК СТРАТЕГИЯ

Если патч ломает существующие тесты:

1. **Откатить STEP 8 (C2)** — duplication removal (может влиять на spatial tests)
2. **Откатить STEP 7 (C1)** — suppress removal (может вскрыть скрытые ошибки)
3. **Откатить STEP 6 (B3)** — SSE activation (может ломать streaming)
4. **Откатить STEP 5 (B2)** — Spatial Oracle (может ломать location detection)
5. **Откатить STEP 4 (B1)** — frontend authority (самый большой, 5 sub-fixes)
6. **Откатить STEP 3 (A3)** — cue normalization
7. **Откатить STEP 2 (A2)** — Dict canonicalization
8. **Откатить STEP 1 (A1)** — GameActionResponse expansion

Откатывать в обратном порядке. Каждый STEP — independently testable.

14. ПРИНЦИПЫ, КОТОРЫЕ ПАТЧ СОБЛЮДАЕТ

1. **Single causal authority** — backend = truth, frontend = renderer
 2. **No silent failures** — все ошибки логируются
 3. **No runtime adapters** — DTO каноничен, конвертация убрана
 4. **No duplication** — один алгоритм в одном месте
 5. **Backward compatibility** — fallback для legacy snapshots
 6. **Atomic migration** — 8 шагов применяются вместе, откатываются вместе
-

15. ФИНАЛЬНАЯ ПРОВЕРКА

1. T3-3 contract repair tests

```
cd backend && python -m pytest tests/test_tz3_contract_repair.py -v
```

2. Existing tests still pass

```
cd backend && python -m pytest tests/ -v --timeout=60
```

```

# 3. Frontend imports OK
cd frontend && python -c "
from api_client import GameActionResponse, HttpGameGateway
from game_screen import GameScreen
from game_loop_bridge import GameLoopBridge
print('Frontend imports OK')
"

# 4. No suppress(Exception) in game_screen
grep -c "suppress(Exception)" frontend/game_screen.py
# Ожидание: 0

# 5. No snapshot_npc_positions_to_dict
grep -rn "snapshot_npc_positions_to_dict" backend/ --include="*.py"
# Ожидание: 0 (только в комментариях)

# 6. NPC positions is Dict
python -c "
import sys; sys.path.insert(0, 'backend')
from app.domain.snapshot import WorldSnapshotDTO
import typing
hints = typing.get_type_hints(WorldSnapshotDTO)
assert hints['npc_positions'].__origin__ == dict
print('npc_positions is Dict OK')
"

```

Если все 6 проверок проходят — патч применён успешно.

16. СЛЕДУЮЩИЙ ШАГ (после применения T3-3)

После подтверждения, что T3-3 применён и тесты проходят, система переходит из **tri-ontology** в **single causal authority**. Следующие слои:

1. **Kernel Isolation Repair** (отдельная инструкция) — закрыть RNG leaks
2. **CK CORE LAYER** (отдельная инструкция) — dynastic simulation systems
3. **Causal Authority Map** — формальная фиксация, кто имеет право изменять/интерпретировать/отображать state

T3-3 — **предусловие** для Kernel Isolation (нельзя чинить kernel, если frontend контаминирует state).

КОНЕЦ ИНСТРУКЦИИ

Документ содержит: - 8 шагов миграции (6 modify + 1 new + 1 test) - 6 atomic patches (A1-A3, B1-B3, C1-C2) - Порядок применения с зависимостями - Критерии приёмки для каждого шага - Стратегию роллбэка - Финальную проверку

Внедрение выполняется разработчиком по этой инструкции.

Документ — ТЗ, не код.